NAMMACONNECT

Technical Development Specification

Context-Aware Kannada Learning and Communication Platform

Document field	Specification
Version	1.0 — MVP implementation baseline
Primary problem statement	Kannada in Everyday Life
Secondary alignment	Kannada for Everyone
Target	KSCST project and hackathon prototype
Platform	Responsive web application
Recommended stack	Next.js + TypeScript, FastAPI, PostgreSQL/SQLite
Status	Development-ready draft; validate content and stack before implementation

Purpose

This document translates the NammaConnect PRD into an actionable engineering plan: architecture, modules, database entities, API contracts, screen behavior, validation rules, implementation phases, testing, and deployment.

1. Product and Engineering Boundaries

NammaConnect helps non-Kannada speakers practise practical conversations for everyday situations in Karnataka. The MVP focuses on verified phrases, scenario-based learning, guided conversation flows, quizzes, bookmarks, basic progress tracking, and offline access to a selected phrase collection.

Engineering principle: ship a dependable learning workflow first. Do not make core functionality dependent on generative AI, live translation APIs, or speech recognition.

1.1 MVP in scope

- Browse categories and scenarios.
- Search by English keywords, scenario names, and supported Kannada text.
- View Kannada text, English meaning, transliteration, contextual notes, and optional pronunciation audio references.
- Complete deterministic, predefined conversation flows.
- Complete quizzes and see feedback and scores.
- Bookmark phrases and track scenario completion.
- Cache a curated phrase collection for offline use.
- Provide a protected content-maintenance workflow, even if initial content is loaded through seed files.

1.2 Explicitly out of scope

- Always-on/background microphone listening.
- Unrestricted AI conversation generation as the only practice method.
- Automatic pronunciation scoring without validated Kannada-specific evaluation.
- Medical advice, diagnosis, or medicine recommendations.
- A complete grammar course, social feed, public user-submitted content, payments, or native mobile apps.
- Claiming the entire application works offline; only cached/downloaded content is expected to work offline.

1.3 Suggested initial content target

Start with four categories (transportation, shopping, education, and basic healthcare communication), 5–10 scenarios per category, 10–15 reviewed phrases per scenario, and at least one conversation plus a short quiz for each scenario. These are planning targets, not existing content counts.

2. Recommended Technology Stack

Layer	Recommended choice	Reason / notes
Web frontend	Next.js + TypeScript	Responsive UI, routing, reusable components, good deployment options.
UI styling	Tailwind CSS	Consistent responsive styling and rapid prototyping.
Backend API	Python + FastAPI	Typed request/response schemas, straightforward REST API, easy testing.
Database	SQLite for local MVP; PostgreSQL for shared deployment	SQLite lowers setup overhead; PostgreSQL is preferable for concurrent multi-user use.
ORM / migrations	SQLAlchemy 2.x + Alembic	Structured persistence and repeatable schema changes.
Validation	Pydantic	Validate API payloads and response models.
Offline web support	Service worker + IndexedDB	Cache app shell and selected phrase/scenario data.
Testing	Pytest + frontend component/e2e tests	Validate API rules and critical user workflows.
Deployment	Frontend and API deployed separately or together	Keep configuration portable; do not hard-code provider-specific secrets.

Keep the initial architecture as a modular monolith: one frontend, one API, and one relational database. Microservices are unnecessary for this scope.

3. System Architecture

```
Browser / Mobile Web
■■■ Next.js UI
■ ■■■ Category & Scenario Explorer
■ ■■■ Phrase Search & Phrase Detail
■ ■■■ Guided Conversation Player
■ ■■■ Quiz & Results
■ ■■■ Bookmarks & Progress
■ ■■■ Offline Cache (Service Worker + IndexedDB)
■
■■■ HTTPS REST API
■■■ FastAPI
■■■ Category / Scenario Module
■■■ Phrase & Search Module
■■■ Conversation Module
■■■ Quiz Module
■■■ Progress / Bookmark Module
■■■ Content Review Module
■■■ Authentication (optional for MVP)
■
■■■ SQLAlchemy → SQLite / PostgreSQL
```

3.1 Request flow

- The frontend requests categories and scenarios from the API.
- The user opens a scenario; the frontend requests its phrases, conversation flow, and quiz metadata.
- The backend returns published, approved content only.
- Quiz answers are evaluated against stored answer keys; feedback is returned with the score.
- Bookmarks and progress are persisted for authenticated users, or stored locally for a no-account prototype.
- The client caches only approved content selected for offline access.

3.2 Content trust boundary

Only approved/published phrases may appear in learner-facing APIs. Draft and under-review content must be visible only to authorized content maintainers. The API must not accept an arbitrary client-supplied flag to mark content as approved.

4. Repository Structure

```
namma-connect/
frontend/
app/
page.tsx
categories/page.tsx
categories/[categoryId]/page.tsx
scenarios/[scenarioId]/page.tsx
phrases/[phraseId]/page.tsx
practice/[scenarioId]/page.tsx
quiz/[scenarioId]/page.tsx
progress/page.tsx
saved/page.tsx
offline/page.tsx
components/
CategoryCard.tsx
ScenarioCard.tsx
PhraseCard.tsx
SearchBar.tsx
ConversationPlayer.tsx
QuizQuestion.tsx
ProgressSummary.tsx
lib/
api.ts
offline-db.ts
types.ts
public/
tests/
backend/
app/
main.py
core/config.py
core/security.py
db/session.py
models/
schemas/
routers/
categories.py
scenarios.py
phrases.py
conversations.py
quizzes.py
progress.py
bookmarks.py
admin_content.py
services/
tests/
alembic/
seed/
categories.json
scenarios.json
phrases.json
conversations.json
quizzes.json
requirements.txt
.env.example
docs/
api-contract.md
content-review-guide.md
testing-plan.md
README.md
```

5. Database Design

Use normalized relational tables. IDs should be UUIDs or stable integer IDs; choose one approach and use it consistently. Timestamps should be stored in UTC. For the simplest prototype, UUIDs are useful when seed content is exchanged between environments.

Table	Important fields	Rules
categories	id, name, description, icon_key, sort_order, status	Unique category name/slug.
scenarios	id, category_id, title, objective, difficulty, duration_minutes, status	category_id references categories.
phrases	id, scenario_id, kannada_text, english_meaning, transliteration, context_note, formality, audio_url, review_status, published_at	Only approved content may be published.
phrase_reviews	id, phrase_id, reviewer_label, status, notes, reviewed_at	Keep review history; avoid storing unnecessary personal details.
conversations	id, scenario_id, title, intro_text, status	One scenario can have multiple flows.
conversation_steps	id, conversation_id, step_order, speaker_role, phrase_id, prompt_text, explanation	Unique (conversation_id, step_order).
conversation_options	id, step_id, label, next_step_id, is_preferred, feedback	Validate that next_step_id belongs to a valid flow.
quiz_questions	id, scenario_id, question_text, question_type, explanation, sort_order, status	Only supported question types accepted.
quiz_options	id, question_id, option_text, is_correct, sort_order	For single-choice questions, enforce one correct answer.
users	id, email or provider_subject, created_at	Optional if accounts are included.
user_progress	id, user_id, scenario_id, completed_at, latest_score, attempts	Unique (user_id, scenario_id) or a separate attempts table.
saved_phrases	user_id, phrase_id, created_at	Composite unique key prevents duplicate bookmarks.

5.1 Data rules

- Use foreign keys and database constraints wherever practical.
- Store the Kannada phrase and its transliteration separately; never generate transliteration by naïve character replacement at render time.
- Use status enums or constrained values for content workflow.
- Keep quiz answer keys on the server; do not include is_correct in the question payload sent before submission.
- Avoid collecting age, precise location, microphone recordings, or other unnecessary personal information.
- Provide seed scripts that can be run repeatedly without duplicating content.

6. API Specification

Base path: /api/v1. All JSON endpoints should use consistent error objects. Public endpoints return published content only. Authentication is optional for browsing, but required for server-synchronized progress and bookmarks.

Method and endpoint	Purpose	Expected behavior
GET /health	Health check	Returns service status; no secrets.
GET /categories	List categories	Published categories sorted by display order.

Method and endpoint	Purpose	Expected behavior
GET /categories/{id}/scenarios	List scenarios	Published scenarios for a category.
GET /scenarios/{id}	Scenario detail	Objective, difficulty, estimated duration.
GET /scenarios/{id}/phrases	Scenario phrases	Approved phrases with meaning and transliteration.
GET /phrases/{id}	Phrase detail	Context, formality, related phrase references.
GET /search?q=...	Search content	Bound query length; return relevant published phrases/scenarios.
GET /scenarios/{id}/conversation	Conversation flow	Return ordered steps/options without hidden answer metadata.
GET /scenarios/{id}/quiz	Quiz questions	Return questions/options, not correct-answer flags.
POST /scenarios/{id}/quiz/submit	Submit quiz	Validate answers, calculate score server-side, return feedback.
GET /me/bookmarks	List saved phrases	Authenticated user only.
POST /me/bookmarks/{phrase_id}	Save phrase	Idempotent; prevent duplicates.
DELETE /me/bookmarks/{phrase_id}	Remove phrase	Return success if already absent.
GET /me/progress	Progress summary	Authenticated user only.
PUT /me/progress/{scenario_id}	Update progress	Validate completion and score values.
GET /offline/manifest	Offline content manifest	List versioned content pack and assets approved for caching.
GET /admin/content	Review content	Admin role required; drafts and review status included.
POST /admin/content/phrases	Create phrase	Admin role required; default status is draft.
PATCH /admin/content/phrases/{id}	Edit phrase	Admin role required; record changes.
POST /admin/content/phrases/{id}/publish	Publish phrase	Require approved review state.

6.1 Example response: phrase

```
{
"id": "phrase_001",
"scenario_id": "scenario_directions",
"kannada_text": "<reviewed Kannada sentence>",
"english_meaning": "Where is the bus stand?",
"transliteration": "<reviewed Latin transliteration>",
"context_note": "A simple question used to ask for a location.",
"formality": "neutral",
"audio_url": null
}
```

Do not use the placeholder values above as published Kannada content. Replace them with phrases reviewed by fluent Kannada speakers.

6.2 Example quiz submission

```
POST /api/v1/scenarios/scenario_directions/quiz/submit

Request:
{
"answers": [
{"question_id": "q1", "selected_option_id": "opt2"},
{"question_id": "q2", "selected_option_id": "opt4"}
]
}

Response:
{
"score": 1,
"total": 2,
"percentage": 50,
"results": [
{"question_id": "q1", "correct": true, "feedback": "Correct."},
{"question_id": "q2", "correct": false, "feedback": "Review the phrase meaning."}
]
}
```

6.3 API error format

```
{
"error": {
"code": "NOT_FOUND",
"message": "The requested published content was not found.",
"request_id": "server-generated-id"
}
}
```

- Use 400 for malformed input, 401 for unauthenticated requests, 403 for insufficient permissions, 404 for unavailable content, 409 for conflicts, and 500 for unexpected errors.
- Do not expose stack traces, database details, access tokens, or secret values to clients.
- Apply rate limits to authentication, search, and content-management endpoints when publicly deployed.

7. Screen-by-Screen UI Behavior

7.1 Home

- Show a concise purpose statement and search field.
- Display category cards and a quick-access area for saved or recently visited content.
- For signed-out users, show a useful browse-first experience; do not block learning behind registration.
- Empty-state copy should explain how to start, not merely show a blank panel.

7.2 Category and Scenario Lists

- Category cards open the scenario list.
- Scenario cards show title, task objective, difficulty, and approximate duration.
- Only published content appears.
- Provide loading, error, and no-content states.

7.3 Phrase Detail

- Display Kannada text prominently, with English meaning and transliteration nearby.
- Show context and politeness notes when reviewed.
- Show a play control only if a valid audio asset or supported text-to-speech service is configured.
- Allow bookmarking and navigation to related phrases.

- Use semantic HTML and verify Kannada font rendering on Android, Windows, and common mobile browsers.

7.4 Guided Conversation Player

- Display one conversational turn at a time.
- Present response choices as clearly labelled buttons.
- On selection, show feedback and continue to the next step.
- Allow retry, restart, and exit.
- Prevent dead ends: every reachable non-terminal step must have a valid next step or a defined completion state.

7.5 Quiz and Results

- Show one question at a time or a clearly grouped short quiz.
- Require an explicit submit action or define immediate-feedback behavior consistently.
- After submission, show score, correct/incorrect feedback, and a link to review relevant phrases.
- Prevent duplicate submissions from incrementing attempts accidentally unless retries are intended.

7.6 Progress

- Show completed scenarios, latest quiz performance, and suggested revision.
- Clearly distinguish local-only progress from synchronized account progress.
- Do not invent streaks or mastery levels without defined rules.

7.7 Offline Phrasebook

- Show the last successful content-pack update and available offline categories.
- Cache a versioned content manifest and selected data in IndexedDB.
- Provide an explicit refresh action while online.
- Handle stale content gracefully; do not silently claim content is current.

8. Core Business Logic

8.1 Search

- Trim whitespace and normalize case for English search.
- Use database full-text search or a simple indexed search for the first dataset.
- Search Kannada text using Unicode-safe storage and query handling.
- Limit query length and result count.
- Rank exact phrase or title matches above broad description matches.

8.2 Quiz scoring

```
score = number_of_correct_answers
total = number_of_questions
percentage = round((score / total) * 100) if total > 0 else 0
```

The server must validate that submitted question and option IDs belong to the requested quiz. Ignore or reject duplicate question IDs and unknown options.

8.3 Conversation flow

- A conversation has an explicit start step and a terminal step.
- Each option points to a valid next step or a terminal state.
- Store feedback for each response path.

- Validate flows during content publication to detect cycles that cannot be exited, missing steps, or unreachable steps.

8.4 Content publication

- New phrases start as draft.
- A reviewer records approval or requests changes.
- Only approved phrases may be published.
- Editing a published phrase should return it to review if the change affects language content.
- Keep a minimal audit trail for publication and major edits.

9. Authentication, Privacy, and Security

The MVP may launch without accounts for basic learning. If server-side bookmarks or progress synchronization are required, add authentication using a well-supported provider or secure session implementation.

- Never store plaintext passwords.
- Use secure, HttpOnly cookies for session tokens when applicable; configure SameSite and Secure correctly in production.
- Enforce authorization on every admin and user-specific endpoint.
- Validate request bodies and constrain string lengths.
- Use environment variables for secrets; commit only a sanitized .env.example.
- Use HTTPS in production.
- Do not record microphone audio by default. If voice features are added, disclose processing and retention behavior.
- Collect only data required for a clearly stated product function.

10. Offline Implementation

Offline support should be narrow and testable. Cache the app shell and a curated phrase pack, not every API response indiscriminately.

- Use a service worker to cache static assets and the offline route.
- Use IndexedDB for structured categories, scenarios, and phrases.
- Use a versioned manifest with pack_version, updated_at, and content IDs.
- When online, fetch the latest manifest and update the local pack safely.
- When offline, use local content and display an offline status indicator.
- Do not cache private user data in shared or unsafe contexts without a deliberate security design.
- Test first-use behavior: the user must download/cache the pack while online before offline access is available.

11. Testing Strategy

Test layer	What to test	Release gate
Unit tests	Quiz scoring, content status rules, search normalization, flow validation.	Core logic tests pass.
API tests	Status codes, schema validation, permissions, published-content filtering.	No draft content leaks publicly.
Database tests	Foreign keys, unique bookmarks, seed idempotency, migrations.	Fresh database builds correctly.
Frontend tests	Category navigation, phrase rendering, quiz feedback, bookmarks.	Critical interactions work.
End-to-end	Home → category → scenario → phrase → conversation → quiz → result.	Happy path completes on mobile and desktop.
Offline tests	Load downloaded phrase pack after disconnect; handle stale pack.	Selected phrases remain available.
Language QA	Kannada correctness, naturalness, transliteration, context and politeness.	Published phrases approved by reviewers.
Usability study	Can beginners find and use phrases? Do task completion and comprehension improve?	Findings documented; no unsupported impact claims.

11.1 Minimum test cases

- A draft phrase never appears in public phrase endpoints.
- A user cannot access another user's progress or bookmarks.
- Submitting an invalid quiz option returns a controlled validation error.
- A conversation with a missing next step cannot be published.
- A phrase search with no matches shows an empty state.
- The phrase collection works after the device goes offline, if it was cached.
- The application remains usable at narrow mobile viewport widths.
- Kannada characters do not appear as empty squares on supported test devices.

12. Implementation Sequence

Sprint	Work	Deliverable
Sprint 0 — Setup	Create repository, environment configuration, linting, database connection, README.	Runnable frontend and API skeleton.
Sprint 1 — Content	Define models, migrations, seed loader, initial approved content format.	Categories, scenarios, and phrases accessible via API.
Sprint 2 — Browse/Search	Build home, categories, scenario list, phrase detail, search.	Users can find and read phrases.
Sprint 3 — Practice	Implement conversation flow and quiz APIs/UI.	Users can practise and receive feedback.
Sprint 4 — Progress	Bookmarks, progress model, optional account flow.	Saved phrases and completion tracking.
Sprint 5 — Offline/QA	Offline content pack, responsive QA, accessibility and language review.	Demonstrable MVP.
Sprint 6 — Evaluation	Run task-based user study, measure outcomes, fix high-impact issues.	Evidence and project demonstration materials.

Sprint duration should be chosen based on team size and available time. Do not commit to a fixed calendar until the content volume, team responsibilities, and external review availability are known.

13. Deployment and Environment

13.1 Development environment

- Use separate frontend and backend environment configuration.
- Provide a repeatable database migration and seed command.
- Include a health endpoint and structured application logs.
- Pin dependencies and document supported runtime versions.

13.2 Environment variables

```
# Backend .env.example
APP_ENV=development
DATABASE_URL=sqlite:///./namma_connect.db
CORS_ORIGINS=http://localhost:3000
SECRET_KEY=replace-with-a-long-random-secret
ADMIN_BOOTSTRAP_ENABLED=false

# Frontend .env.example
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000/api/v1
```

Never use the example secret in production. Keep private credentials out of source control and do not expose backend secrets through NEXT_PUBLIC variables.

13.3 Production checklist

- HTTPS enabled.
- Production database configured and backed up.
- Migrations run as part of controlled deployment.
- CORS limited to approved origins.
- Admin account provisioned securely.
- Error reporting and health monitoring enabled.
- Privacy notice and contact details published.

- Seeded content reviewed before release.

14. Definition of Done

- The core user journey works end-to-end on desktop and mobile.
- At least one fully reviewed scenario exists in each initial category.
- Published phrases have approved Kannada text, meaning, transliteration, and context.
- Conversation flows and quiz scoring pass automated tests.
- Public endpoints expose published content only.
- Bookmarks and progress work according to the chosen account/local-storage design.
- The selected offline phrase pack works after being cached.
- Accessibility, error states, and Kannada rendering have been checked.
- A small user evaluation has been completed and documented.
- Known limitations and future work are clearly listed.

15. Decisions to Lock Before Coding

Decision	Recommended default	Why it matters
Frontend	Next.js + TypeScript	UI structure and routing.
Backend	FastAPI	API contracts and Python ecosystem.
Database	SQLite locally; PostgreSQL for shared deployment	Data model and deployment configuration.
Authentication	No account required for browsing; optional accounts for sync	Avoid blocking initial use.
Language content	Human-reviewed seeded content	Correctness is central to product trust.
Conversation input	Tap-to-select choices for MVP	Avoid unreliable speech dependencies.
Voice	Optional text-to-speech only after validation	Keep initial release feasible.
Offline scope	Curated phrase pack	Makes offline support testable.
Admin tooling	Minimal protected content-review screen or controlled seed workflow	Prevents unreviewed phrases being published.

Recommended next engineering artifact: a task-by-task implementation backlog with acceptance criteria and a first batch of reviewed scenario/phrase content. Build the data and learning flow before integrating AI or speech.